Software Testing

Software Testing

- Process to identify the correctness, completeness and quality of the developed software.
- Helps in identifying errors, gaps or missing requirements.
- Can be done manually or using Automation tools

A more appropriate definition is:

"Testing is the process of executing a program with the intent of finding errors."

Why should We Test ?

Although software testing is itself an expensive activity, yet launching of software without testing may lead to cost potentially much higher than that of testing, specially in systems where human safety is involved.

China Airbus crashing, 1994

1985, Canada

Windows 98 crashes live on CNN

The real reason Boeing's new plane crashed twice

Who should Do the Testing ?

- Testing requires the developers to find errors from their software
- It is difficult for software developer to point out errors from own creations.
- Many organisations have made a distinction between development and testing phase by making different people responsible for each phase.

(a) Developer
Understands the system
but will test "gently"
and is driven by "delivery"

(b) Independent tester
Must learn about the system,
but will attempt to break it
and is driven by quality

Testing Principles

1. All tests should be traceable to customer requirements
2. Tests should be planned long before testing begins.
3. The Pareto principle applies to software testing
4. Testing should begin “in the small” and progress toward testing “in the large.”
5. Exhaustive testing is not possible
6. To be most effective, testing should be conducted by an independent third party.

Error, Mistake, Bug, Fault and Failure

People make **errors**. A good synonym is **mistake**. This may be a syntax error or misunderstanding of specifications. Sometimes, there are logical errors.

When developers make mistakes while coding, we call these mistakes "**bugs**".

A **fault** is the representation of an error, where representation is the mode of expression, such as narrative text, data flow diagrams, ER diagrams, source code etc. Defect is a good synonym for fault.

A **failure** occurs when a fault executes. A particular fault may cause different failures, depending on how it has been exercised.

Test, Test Case and Test Suite

- **Test** and **Test case** terms are used interchangeably.
- Test case describes an input description and an expected output description.
- If the expected output and observed output are different from each other, then there is a **failure** and it must be recorded properly.
- The set of test cases is called a **test suite**. Hence any combination of test cases may generate a test suite.

Test Case ID	
Section-I (Before Execution)	Section-II (After Execution)
Purpose :	Execution History:
Pre condition: (If any)	Result:
Inputs:	If fails, any possible reason (Optional);
Expected Outputs:	Any other observation:
Post conditions:	Any suggestion:
Written by:	Run by:
Date:	Date:

: Test case template

Verification and Validation

- Verification: “Does the product meet its specifications?”.

  Verification is the process of evaluating a system or component to determine whether the products of a given development phase satisfy the conditions imposed at the start of that phase.

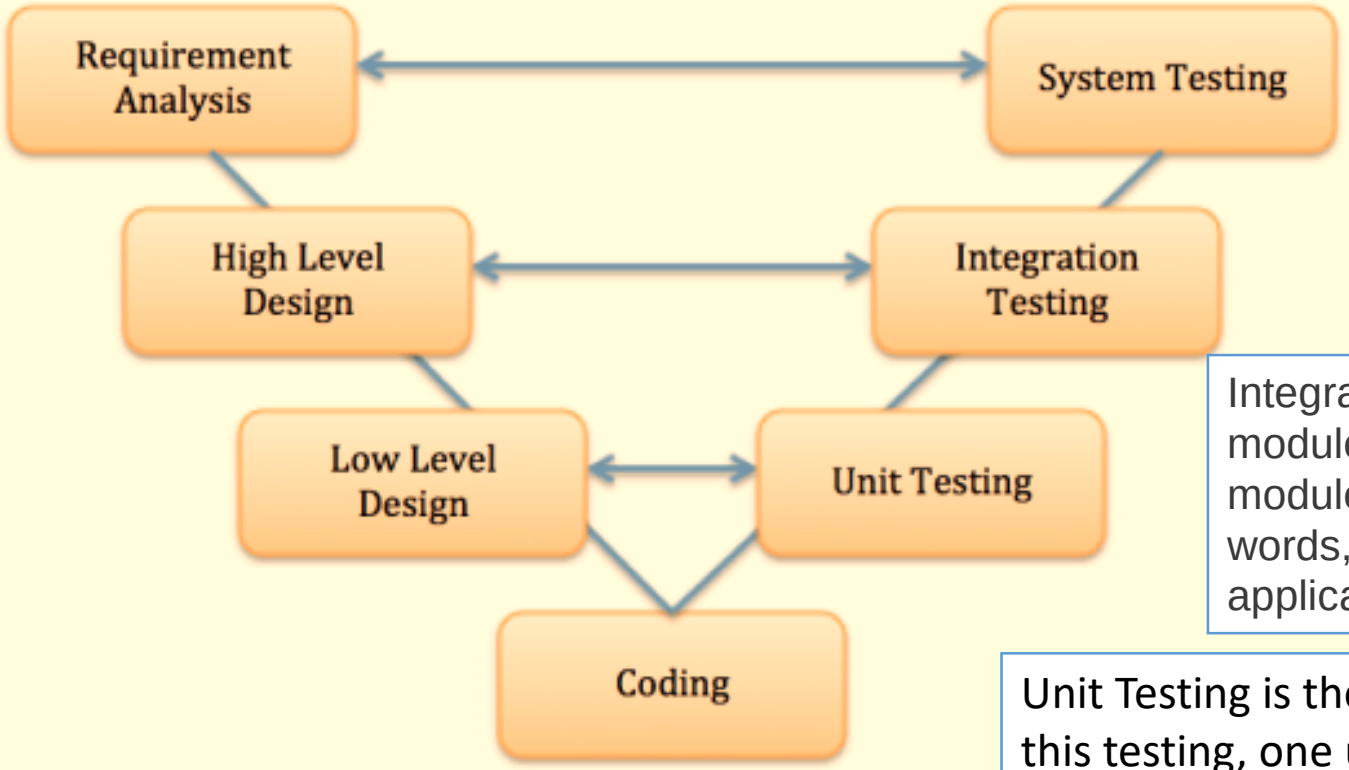
- Validation: “Does the product perform as desired?”

  Validation is the process of evaluating a system or component during or at the end of development process to determine whether it satisfies the specified requirements .

Testing= Verification+Validation

V Model of STLC vs SDLC

Levels of Testing

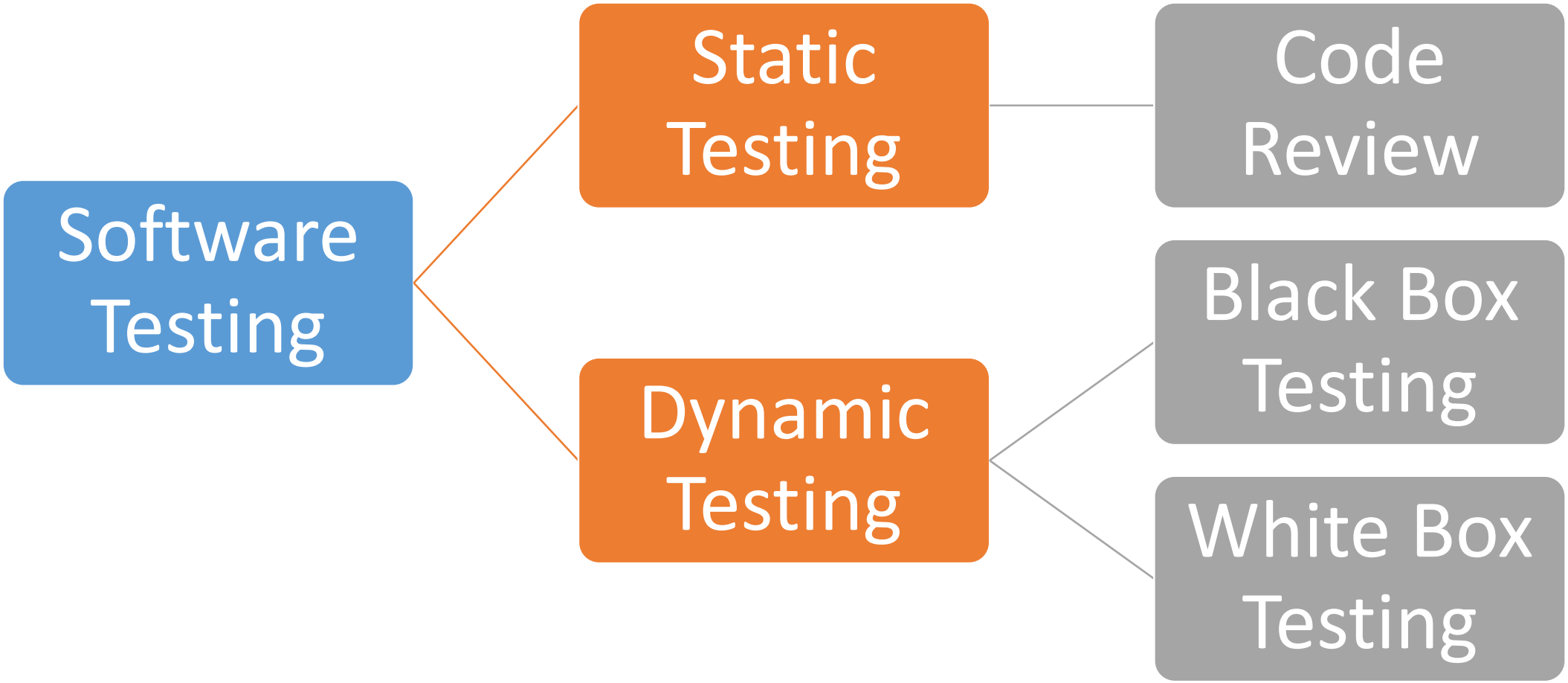


In System Testing, all the system test cases, functional test cases and non-functional test cases are executed. In other words, the actual and full fledge testing of the application takes place here. Defects are logged and tracked for its closure.

Integration testing is a technique where the unit tested modules are integrated and tested whether the integrated modules are rendering the expected results. In simpler words, It validates whether the components of the application work together as expected.

Unit Testing is the process of testing a module, at a time. In this testing, one unit, that is the smallest functional unit of the software is tested at a time. To perform a unit test, unit test cases are developed and is performed by the developer or the tester who has good command over programming.

Types of Testing



Code Review

- Code review for a model is carried out after the module is successfully compiled and the all the syntax errors have been eliminated.
- Code reviews are extremely cost-effective strategies for reduction in coding errors and to produce high quality code.
- Normally, two types of reviews are carried out on the code of a module: Code Inspection and Code Walk Through

Code Walk Throughs

- Code walk through is an informal code analysis technique.
- In this technique, after a module has been coded, successfully compiled and all syntax errors eliminated, a few members of the development team are given the code to read and understand code ,few days before the walk through meeting.
- Each member selects some test cases and simulates execution of the code by hand (i.e. trace execution through each statement and function execution).
- The main objectives of the walk through are to discover the algorithmic and logical errors in the code. The members note down their findings to discuss these in a walk through meeting where the coder of the module is present.

Guidelines for the walkthroughs

- The team performing code walk through should not be either too big or too small. Ideally, it should consist of between three to seven members.

- Discussion should focus on discovery of errors and not on how to fix the discovered errors.

- In order to foster cooperation and to avoid the feeling among engineers that they are being evaluated in the code walk through meeting, managers should not attend the walk through meetings.

Code Inspection

- The aim of code inspection is to discover some common types of errors caused due to oversight and improper programming.
- In other words, during code inspection, the code is examined for the presence of certain kinds of errors, in contrast to the hand simulation of code execution done in code walk throughs.
- In addition to the commonly made errors, adherence to coding standards is also checked during code inspection.
- Good software development companies collect statistics regarding different types of errors commonly committed by their engineers and identify the type of errors most frequently committed

Such a list of commonly committed errors can be used during code inspection to look out for possible errors.

- Use of uninitialized variables.
- Jumps into loops.
- Nonterminating loops.
- Incompatible assignments.
- Array indices out of bounds.
- Improper storage allocation and deallocation.
- Mismatches between actual and formal parameter in procedure calls.
- Use of incorrect logical operators or incorrect precedence among operators.
- Improper modification of loop variables.
- Comparison of equally of floating point variables, etc.

White Box Testing

- **White box testing** is a software testing technique where, the internal structure, design, coding and all the events are tested.
- As the name says, white box, which means whatever (that all the internal modules) in the box that is, the software is visible to the tester.
- This testing technique deals with the **verification of how the flow of the input leads to the output.**
- White box testing is referred to by different names. Such as **clear box testing or the Open box testing or structural testing**. White box testing is performed by a set of developers..

Types of White Box testing

- **Statement Coverage**
- **Branch Coverage**
- **Path Coverage**

Statement Coverage

- The statement coverage strategy aims to design test cases so that every statement in a program is executed at least once.
- The principal idea governing the statement coverage strategy is that unless a statement is executed, it is very hard to determine if an error exists in that statement.
- Unless a statement is executed, it is very difficult to observe whether it causes failure due to some illegal memory access, wrong result computation, etc.
- However, executing some statement once and observing that it behaves properly for that input value is no guarantee that it will behave correctly for all input values.

Example: Consider the Euclid's GCD computation algorithm:

```
int compute_gcd(x, y)
     int x, y;
{
    1    while (x! = y){
   2         if (x>y) then
    3                    x= x – y;
   4         else  y= y – x;
   5   }
   6   return x;
}
```

the test set {(x=3, y=3), (x=4, y=3), (x=3, y=4)}.

Branch Coverage

- In the branch coverage-based testing strategy, test cases are designed to make each branch condition to assume true and false values in turn.
- Branch testing is also known as edge testing as in this testing scheme, each edge of a program's control flow graph is traversed at least once.
- The only difference is that in statement coverage all statement has to be covered at least once, on the other hand, in branch coverage, the output of all the branch is checked whether returning true or false, but both should be reached.
- It is obvious that branch testing guarantees statement coverage and thus is a stronger testing strategy compared to the statement coverage-based testing .

{(x=3, y=3), (x=3, y=2), (x=4, y=3), (x=3, y=4)}.

Path coverage

- In this test, it is checked whether all the paths are covered at least once. This is mainly a detailed task to test how a function performed as the data flows from it.
- It mainly checks from the function start to the exit point.
- It is important to note that, both the concept of statement testing and the branch coverage is performed in this testing technique.
- The test cases for this technique is also, designed in such a way that it traverses all loops and reach up to the final output covering all the statements.

Basis Path Testing

- A white box method
- Proposed by McCabe in 1980’s
- A hybrid of path testing and branch testing methods.
- Based on Cyclomatic complexity and uses control flow to establish the path coverage criteria

Cyclomatic Complexity

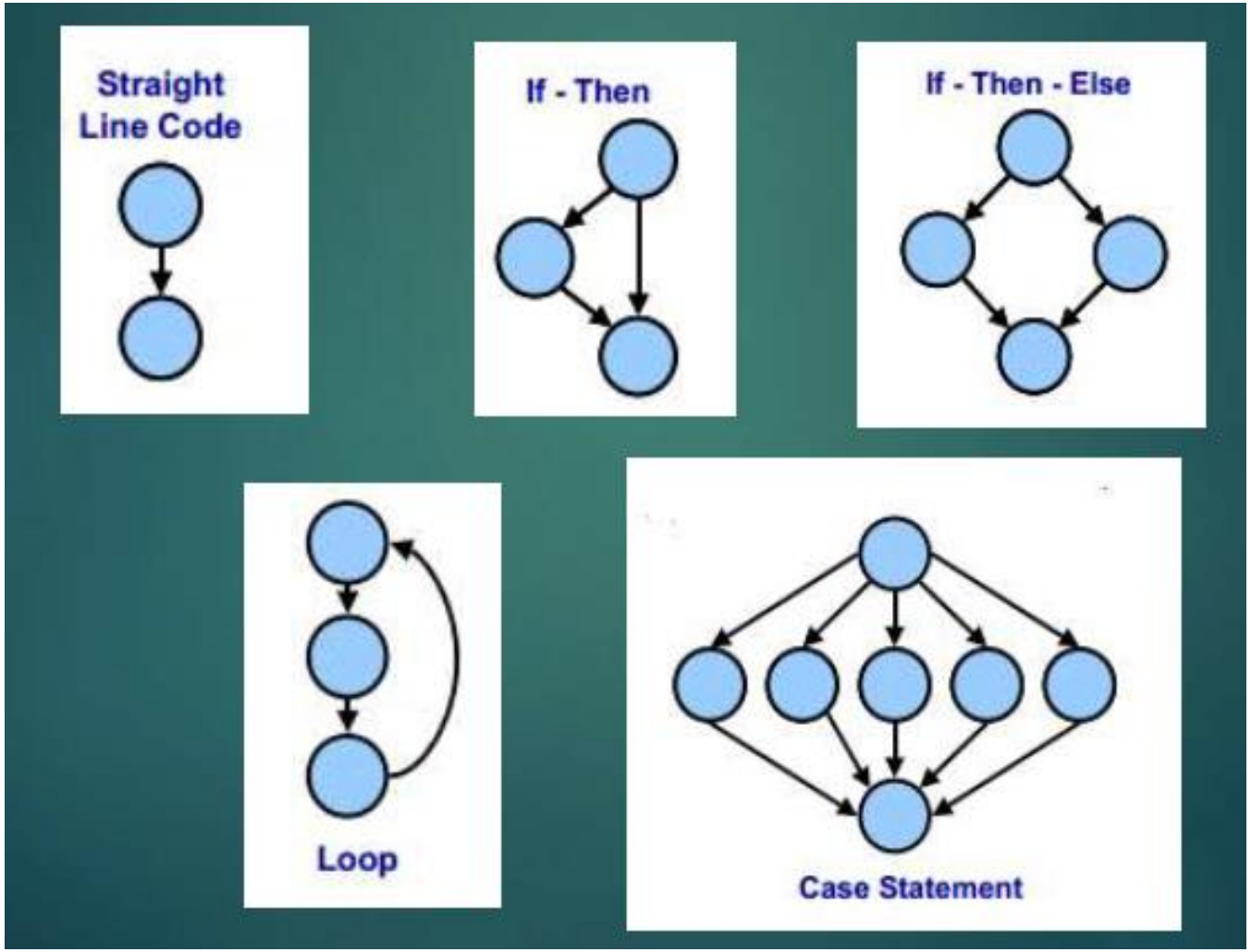
- Cyclomatic Complexity is a software metric for measuring the complexity of a program. This is used for calculation of independent path in a software or a program through which the program control may flow.
- Measures the number of linearly independent paths through a program.
- The higher the number the more complex the code.
- When we say independent path, it means that it is the path which has at least one edge which has not been traversed before any other paths.

Basic path testing approach

- Step 1: Draw a control flow graph.
- Step 2: Determine Cyclomatic complexity.
- Step 3: Find a basis set of paths.
- Step 4: Generate test cases for each path.

Basic Control Flow Graph Structures

- Arrows or edges represent flows of control.
- Circles or nodes represent actions.
- Areas bounded by edges and nodes are called regions.
- A predicate node is a node containing a condition.



1: IF A = 100

2: THEN IF B > C

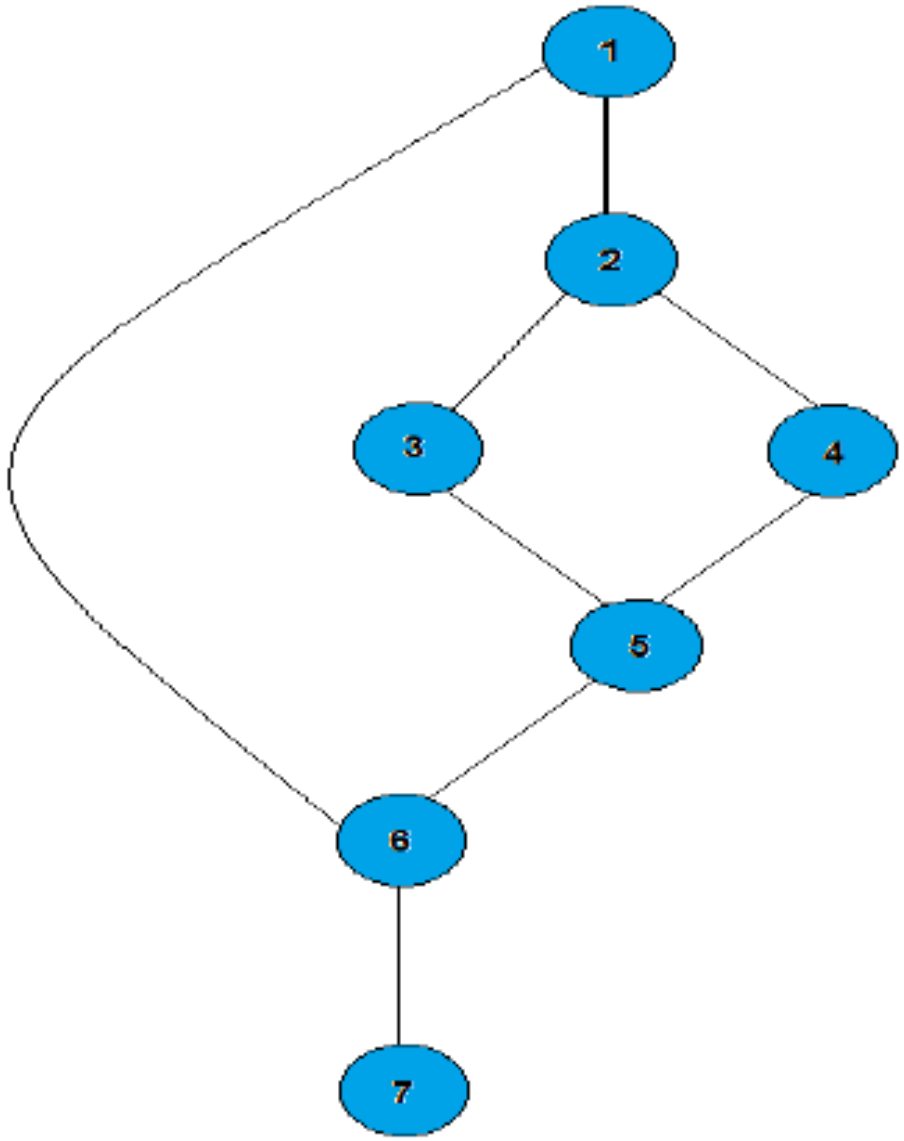
3: THEN A = B

4: ELSEA= C

5: ENDIF

6: ENDIF

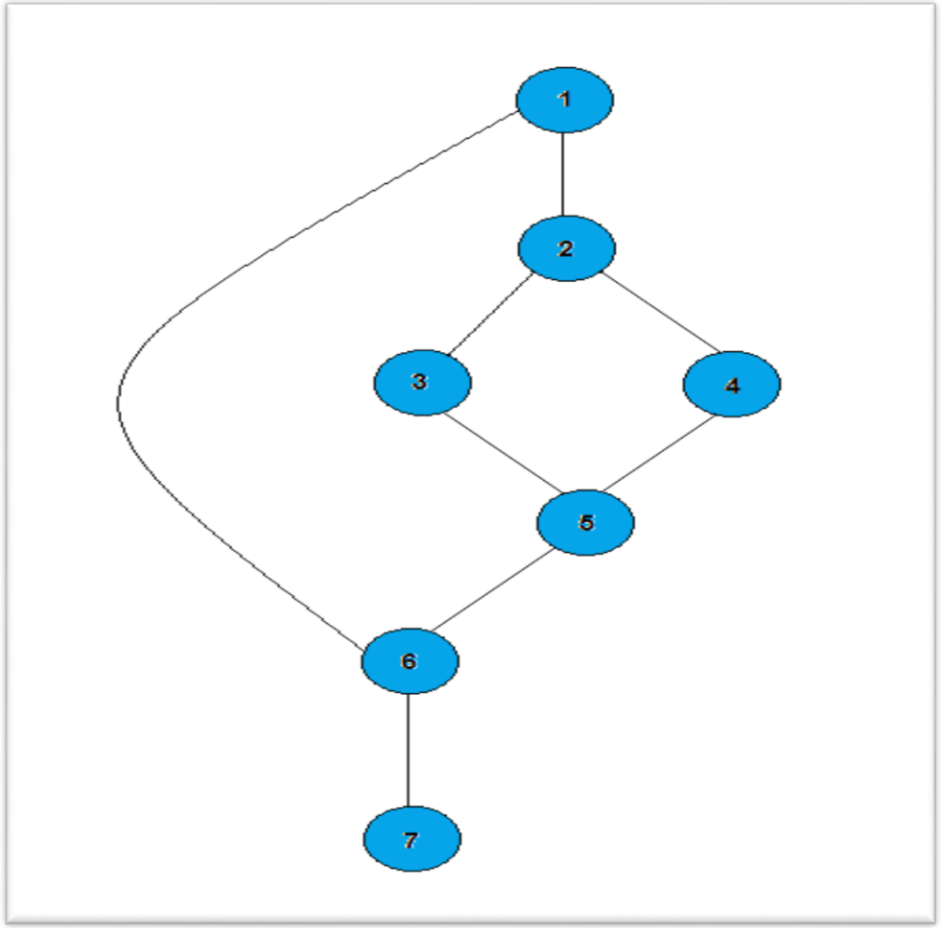
7: Print A



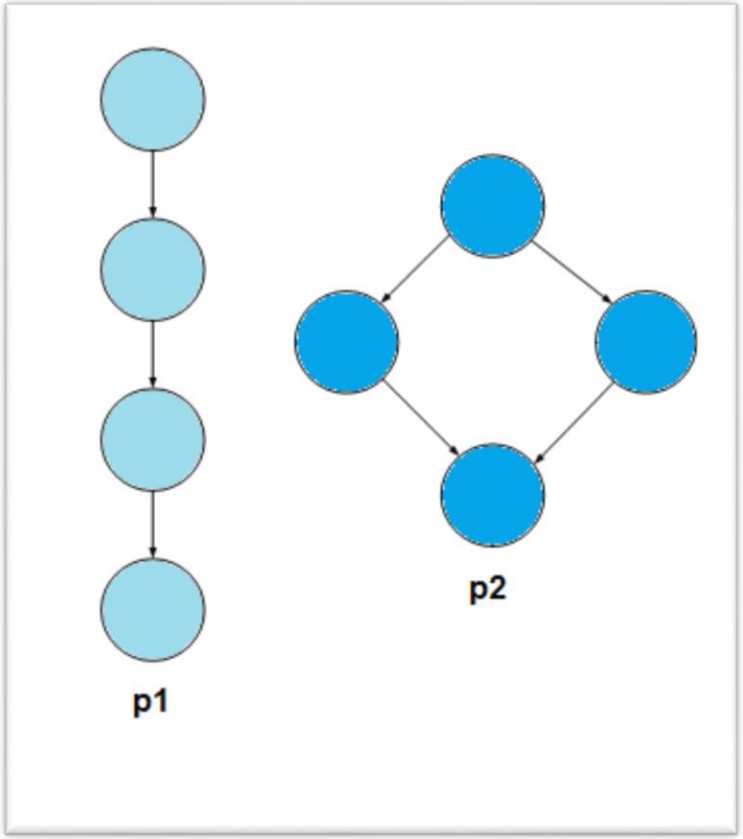
Determine Cyclomatic complexity

- **Cyclomatic complexity = edges - nodes + 2p**
  where p = number of unconnected parts of the graph

- **Cyclomatic complexity= Number of Predicate Nodes + 1**
- **Cyclomatic complexity = number of regions in the control flow graph**



Cyclomatic complexity = 8-7+ 2*1= 3.



Cyclomatic complexity = 7-8+ 2*2= 3.